

ASSIGNMENT-6.3

V. Alok

2303A5300

Batch-46

Task Description #1: Classes (Student Class)

Scenario

You are developing a simple student information management module.

Task

- Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.
- The class should include attributes such as name, roll number, and branch.
- Add a method `display_details()` to print student information.
- Execute the code and verify the output.
- Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

- A Python class with a constructor (`__init__`) and a `display_details()` method.
- Sample object creation and output displayed on the console.
- Brief analysis of AI-generated code.

Prompt used:

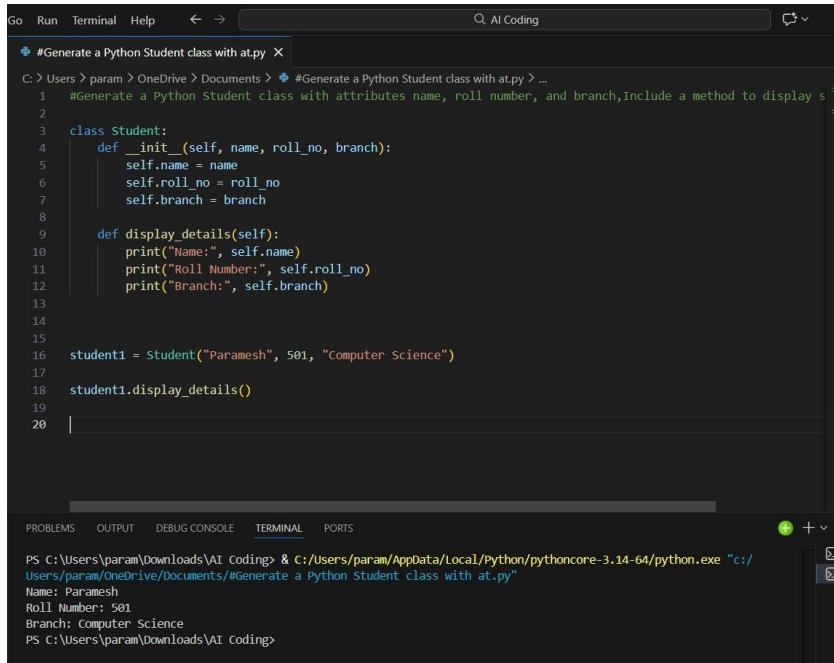
Generate a Python Student class with attributes name, roll number, and branch. Include a method to display student details. Create a sample object and print the output Code :

```
class Student:
    def __init__(self, name,
roll_no, branch):
        self.name = name
        self.roll_no = roll_no
        self.branch =
branch
    def display_details(self):
print("Name:", self.name)
print("Roll
```

```
Number:", self.roll_no)

print("Branch:", self.branch) student1 =
Student("Paramesh", 101, "Computer Science")

student1.display_details() output
:
```



The screenshot shows a VS Code editor window with a file named "#Generate a Python Student class with at.py". The code defines a `Student` class with an `__init__` method that takes `name`, `roll_no`, and `branch` as arguments, and a `display_details` method that prints the student's information. An instance `student1` is created with the values "Paramesh", 501, and "Computer Science". The terminal at the bottom shows the output of running the script, which matches the expected output from the code.

```
#Generate a Python Student class with at.py
C:\Users\param> param > OneDrive > Documents > #Generate a Python Student class with at.py > ...
1 #Generate a Python Student class with attributes name, roll number, and branch, Include a method to display s
2
3 class Student:
4     def __init__(self, name, roll_no, branch):
5         self.name = name
6         self.roll_no = roll_no
7         self.branch = branch
8
9     def display_details(self):
10        print("Name:", self.name)
11        print("Roll Number:", self.roll_no)
12        print("Branch:", self.branch)
13
14
15
16 student1 = Student("Paramesh", 501, "Computer Science")
17
18 student1.display_details()
19
20 |

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\param\Downloads\AI Coding> & C:/Users/param/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/
Users/param/OneDrive/Documents/#Generate a Python Student class with at.py"
Name: Paramesh
Roll Number: 501
Branch: Computer Science
PS C:\Users\param\Downloads\AI Coding>
```

Explanation :

- 1) The AI correctly generated a Python class using a constructor (`__init__`) to initialize student details.
- 2) The `display_details()` method cleanly prints all attributes in a readable format.

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
- Analyze the generated loop logic.

- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

Prompt used:

Generate a Python function that prints the first 10 multiples of a given number using a loop. Then generate the same functionality using a while loop instead of a for loop.

Code :

```
def print_multiples_for(n):
```

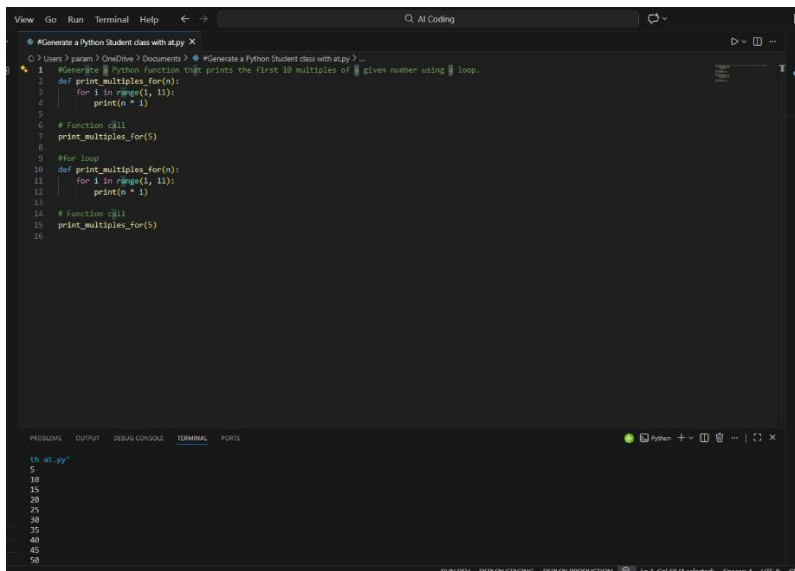
```
    for i in range(1, 11):
```

```
        print(n * i)
```

```
# Function call print_multiples_for(5)
```

output

:



```

1 #Generate a Python function that prints the first 10 multiples of a given number using a loop.
2 def print_multiples_for(n):
3     for i in range(1, 11):
4         print(n * i)
5
6 # function call
7 print_multiples_for(5)
8
9 #for loop
10 def print_multiples_for(n):
11     for i in range(1, 11):
12         print(n * i)
13
14 # function call
15 print_multiples_for(5)
16

```

Terminal Output:

```

In: at.py
5
10
15
20
25
30
35
40
45
50

```

Explanation :

1)The for loop version is more concise and easier to read when the number of iterations is fixed.

2)The while loop provides more control over the loop variable but requires manual incrementing.

3)Both implementations are correct and produce identical output, demonstrating different looping approaches.

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

- Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups(e.g., child, teenager, adult, senior).
- Analyze the generated conditions and logic.
- Ask the AI to generate the same classification using alternative conditional structures (e.g.,simplified conditions or dictionary-based logic).

Expected Output #3

- A Python function that classifies age into appropriate groups.
- Clear and correct conditional logic.
- Explanation of how the conditions work.

Prompt used:

Generate a Python function using nested if-elif-else statements to classify age into child, teenager, adult, and senior.

Then generate the same classification using a simplified or alternative

conditional structure Code : `def classify_age_nested(age):` if age < 0:

```
    return "Invalid age" elif
```

```
age < 13:    return "Child"
```

```
elif age < 20:    return
```

```
"Teenager" elif age < 65:
```

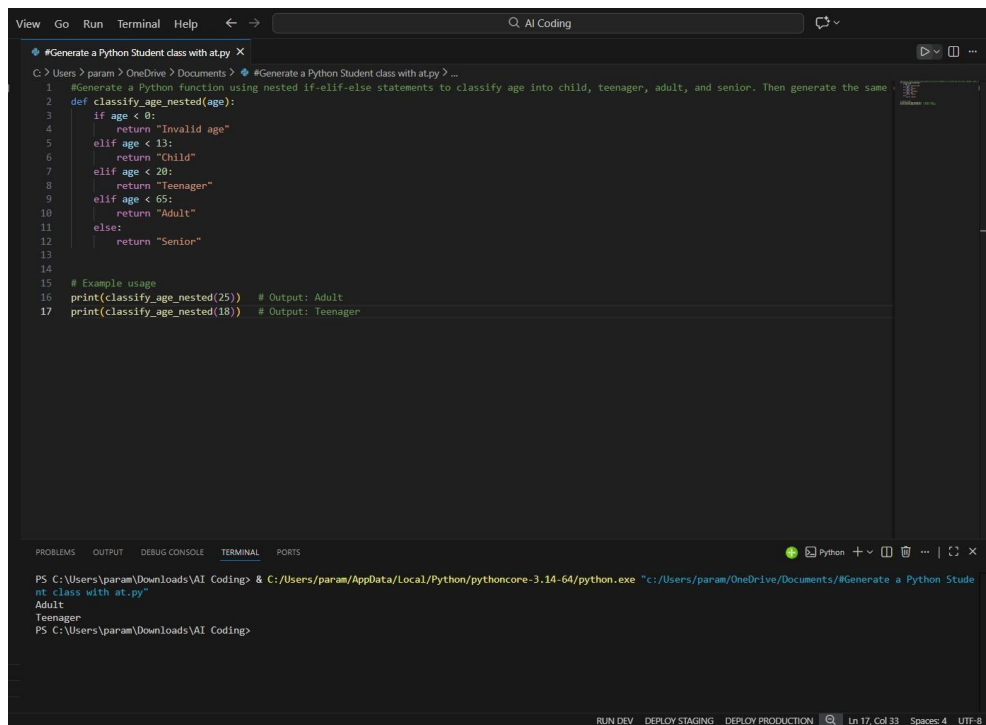
```
return "Adult" else:
```

```
return "Senior"
```

```
# Example usage print(classify_age_nested(25)) #
```

Output: Adult print(classify_age_nested(18)) # Output:

Teenager output :



The screenshot shows a VS Code editor window with a Python file named `at.py`. The code defines a function `classify_age_nested(age)` that uses nested if-elif-else statements to categorize an age into 'Invalid age', 'Child', 'Teenager', 'Adult', or 'Senior'. Below the function, there are example usage lines: `print(classify_age_nested(25))` (commented as # Output: Adult) and `print(classify_age_nested(18))` (commented as # Output: Teenager). The terminal at the bottom shows the command prompt running these lines, resulting in the output 'Adult' and 'Teenager'.

```
1 #Generate a Python function using nested if-elif-else statements to classify age into child, teenager, adult, and senior. Then generate the same
2 def classify_age_nested(age):
3     if age < 0:
4         return "Invalid age"
5     elif age < 13:
6         return "Child"
7     elif age < 20:
8         return "Teenager"
9     elif age < 65:
10        return "Adult"
11    else:
12        return "Senior"
13
14
15 # Example usage
16 print(classify_age_nested(25)) # Output: Adult
17 print(classify_age_nested(18)) # Output: Teenager
```

```
PS C:\Users\param\Downloads\VAI Coding> & C:/Users/param/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/param/OneDrive/Documents/#Generate a Python Student class with at.py"
Adult
Teenager
PS C:\Users\param\Downloads\VAI Coding>
```

Explanation :

- 1) The if-elif-else structure checks age ranges step by step and assigns the correct category.
- 2) The simplified approach uses a list of limits, making the logic more compact and easier to modify.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

- Use AI assistance to generate a `sum_to_n()` function using a for loop.
- Analyze the generated code.
- Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

- Python function to compute the sum of first n numbers.

- Correct output for sample inputs.
- Explanation and comparison of different approaches [Prompt used :](#)

Generate a Python function to calculate the sum of the first n natural numbers using a for loop.

Analyze the logic and also provide an alternative implementation using a while loop or a mathematical formula. Code : `def sum_natural_for(n):`

```
    total = 0
    for i in range(1,
n + 1):
```

```
        total += i
    return total
```

`def sum_natural_while(n):`

```
    total = 0
    i = 1
    while i <= n:
total += i
    i += 1
```

`return total` `def sum_natural_formula(n):`

```
    return n * (n + 1) // 2
```

Example usage:

```
print(sum_natural_for(5)) # Output: 15
print(sum_natural_while(5))
```

```
# Output: 15
print(sum_natural_formula(5)) # Output: 15
```

```

1 # Generate a Python function to calculate the sum of the first n natural numbers using a for loop. Analyze the logic and also provide an alternative
2 # implementation using a while loop or a mathematical formula.
3
4 def sum_natural_for(n):
5     total = 0
6     for i in range(1, n + 1):
7         total += i
8     return total
9
10 def sum_natural_while(n):
11     total = 0
12     i = 1
13     while i <= n:
14         total += i
15         i += 1
16     return total
17
18 def sum_natural_formula(n):
19     return n * (n + 1) // 2
20
21 # Example usage:
22 print(sum_natural_for(5)) # Output: 15
23 print(sum_natural_while(5)) # Output: 15
24 print(sum_natural_formula(5)) # Output: 15

```

Explanation :

- 1) The for loop approach adds numbers sequentially and is easy to understand for beginners.
- 2) The while loop gives manual control over iteration but requires careful incrementing.

3)The mathematical formula is the most efficient method with constant time complexity

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

- Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(),and check_balance().
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5

- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation.

Prompt used:

Generate a Python class for a Bank Account with methods to deposit, withdraw, and check balance. Make sure the class keeps track of the balance and handles cases like insufficient funds. Also, add comments to explain the logic Code :

```
class BankAccount:
    def __init__(self, owner, initial_balance=0):
        """
        Initializes the bank account with the owner's name and an initial balance.
        """
        self.owner = owner
        self.balance = initial_balance

    def deposit(self, amount):
        """
        Deposits a specified amount into the account.
        """
```

```

        """      if amount > 0:          self.balance += amount
print(f"Deposited: {amount}.
New balance: {self.balance}")      else:
        print("Deposit amount must be positive.")

def withdraw(self, amount):
    """
    Withdraws a specified amount from the account if sufficient funds are available.
    """
    if amount > 0:
if amount <= self.balance:
self.balance -= amount
print(f"Withdrew:
{amount}.
New balance: {self.balance}")      else:

        print("Insufficient funds. Withdrawal denied.")
else:      print("Withdrawal amount must be
positive.")

def check_balance(self):
    """
    Returns the current balance of the account.
    """

    print(f"Current balance: {self.balance}")
return self.balance # Demonstration of usage account
= BankAccount("Paramesh", 1000)
# Deposit operation account.deposit(500) #
Expected balance: 1500

```

Withdrawal operation account.withdraw(200) # Expected balance:

1300 # Attempt to withdraw more than balance

account.withdraw(1500) # Should deny due to insufficient funds

Check balance account.check_balance() # Should

show 1300 output:

```
Python 3.10.12 Shell
# BankAccount class with deposit, withdraw, and check_balance methods.
class BankAccount:
    def __init__(self, owner, initial_balance):
        """Initialize the bank account with the owner's name and an initial balance."""
        self.owner = owner
        self.balance = initial_balance

    def deposit(self, amount):
        """Deposit a specified amount into the account.
        If amount is 0 or negative, it will not be added.
        """
        if amount > 0:
            self.balance += amount
            print(f"Deposited {amount}. New balance: {self.balance}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        """Withdraw a specified amount from the account if sufficient funds are available.
        If amount is 0 or negative, it will not be withdrawn.
        If amount is greater than the current balance, the withdrawal is denied.
        """
        if amount > 0:
            if self.balance >= amount:
                self.balance -= amount
                print(f"Withdrew {amount}. New balance: {self.balance}")
            else:
                print("Insufficient funds. Withdrawal denied.")
        else:
            print("Withdrawal amount must be positive.")

    def check_balance(self):
        """Check the current balance of the account.
        Returns the current balance.
        """
        print(f"Current balance: {self.balance}")
        return self.balance

# Demonstration of usage
account = BankAccount("Vernon", 1000)

# Deposit operation
account.deposit(300) # Expected balance: 1300

# Withdrawal operation
account.withdraw(200) # Expected balance: 1100

# Attempt to withdraw more than balance
account.withdraw(1500) # Should show due to insufficient funds

# Check balance
account.check_balance() # Should show 1100
```

Explanation:

- 1)The BankAccount class initializes with an owner name and an initial balance, ensuring each account is uniquely identified.
- 2)The deposit() method checks for positive amounts before adding them to the balance, ensuring basic validation.
- 3)The withdraw() method ensures that the withdrawal amount is positive and that the account has sufficient funds before completing the transaction. If not, it prevents the withdrawal.
- 4)The check_balance() method simply returns and prints the current balance, giving a clear snapshot of account funds.